

SOA SEMINAR - GMAIL SMTP

Gmail SMTP with OAuth 2.0

SMTP + OAuth 2.0 with FastAPI and Google Cloud

524K0012 - Nguyễn Đôn Thịnh

524K0005 - Vũ Văn Minh Hiếu

524K0034 - Lâm Gia Vũ

What are Gmail SMTP and OAuth 2.0?

Gmail SMTP

Google's mail server that supports **SMTP**, the standard protocol for sending email between providers.

helps sending email through different mail providers

OAuth 2.0

a modern authorization method that grants an app a limited-usage token so the app can have limited scope on your gmail

Together: the app logs in to Gmail SMTP with an **OAuth token** (XOAUTH2), not a password.

APPLICATIONS

When Gmail SMTP + OAuth 2.0 fits



Approval workflows

Leave, purchase or expense requests answered from Gmail



Helpdesk & CRM

Staff reply to customers from their own address



School tools

Teachers email students from their TDTU account



Email clients

Thunderbird and Outlook sign in to Gmail with OAuth

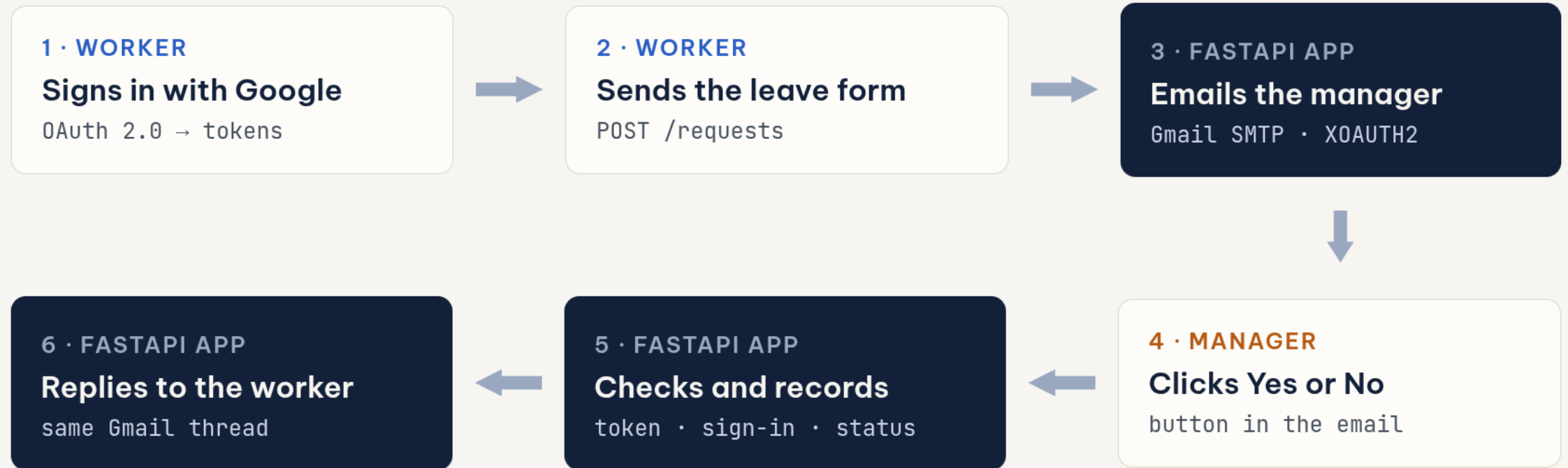
WHY USE IT

Mail comes from the user's real address
No password: a limited, revocable token
No mail server to run: Google delivers it

WHEN NOT TO USE IT

App's own address or bulk mail → SendGrid, Amazon SES
Send-only access → Gmail API with gmail.send
Gmail limit: ~500 recipients/day personal, ~2,000 Workspace

THE DEMO WORKFLOW



OAuth2.0 responsible for signing in

WHY USE

OAuth 2.0

OAuth 2.0 · AUTHORIZATION

What can the app do?

An access token: limited permission to send email as you.

SCOPES IN THE TOKEN

What exactly is allowed?

<code>openid</code>	sign the user in
<code>email</code>	read their email address
<code>mail.google.com</code>	send email as the user

You sign in on **Google's page**. The app never takes the real password

Registering our app with Google

Branding

App name on the consent screen

Audience(internal or external, range of accepted users)

Internal: only TDTU accounts

Data Access

openid · email · mail.google.com

Client

Client ID (public) · Client Secret (server only)

Redirect URI: the only address Google sends the result to

<http://localhost:8002/auth/callback>

What the user sees when signing in

The illustration shows a Google sign-in consent screen. At the top, it says 'Sign in with Google'. Below this is a header section (1) with a blue circle containing 'AD' and the text 'Absence Demo'. The main text says 'wants to access your Google Account'. Below that is a section (3) showing the email address 'sv@student.tdtu.edu.vn'. The largest section (2) is titled 'This will allow Absence Demo to:' and contains a list of permissions: 'See your primary Google Account email address', 'Associate you with your personal info on Google', and 'Read, compose, send, and permanently delete all your email from Gmail'. At the bottom are 'Cancel' and 'Allow' buttons.

Sign in with Google

AD Absence Demo

wants to access your Google Account

sv@student.tdtu.edu.vn

This will allow Absence Demo to:

- See your primary Google Account email address
- Associate you with your personal info on Google
- Read, compose, send, and permanently delete all your email from Gmail

Cancel Allow

1 Branding

App name, logo, support email and privacy links, set on the Branding page

2 Scopes

One line per scope declared in Data Access

email · openid · mail.google.com

3 Account

Audience is Internal, so only TDTU accounts can sign in

AUTHORIZATION CODE FLOW

Google's flow, mapped to our code



- 1 Request** · browser sent to Google
`/login → authorization\_url\(\)`
- 2 Login & consent** · on Google's page
none of our code runs here
- 3 One-time code** · back to our app
`/auth/callback?code=...&state=...`
- 4 Exchange** · code + secret → tokens, email
`exchange\_code\(\)`
- 5 Use token** · Gmail SMTP with XOAUTH2
`send\_mail\(\)`
- + Refresh** · new token after ~1 hour
`access\_token\_for\(\)`